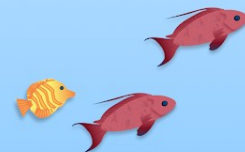


Wakacyjne Wyzwanie

Z  KN Solvro

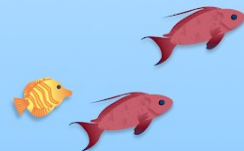


Wykład 7

VPN: Wireguard

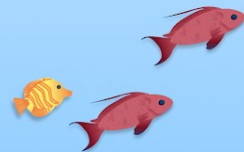


Plan na dziś



- Czym jest VPN?
- Jak działa Wireguard?
- Jak skonfigurować Wireguard?
- Wireguard vs stanowy firewall



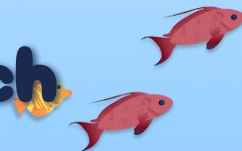


Rozdział 1

Czym jest VPN?



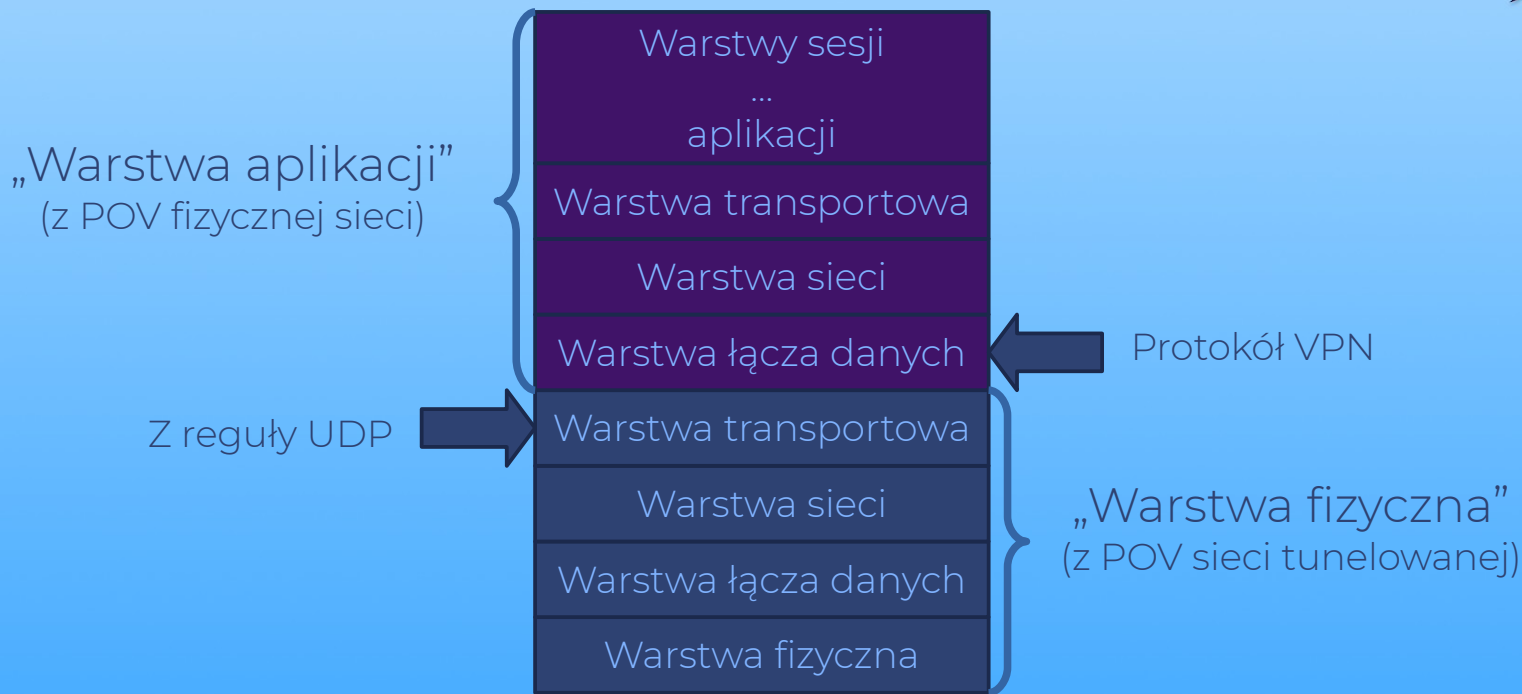
Ograniczenia fizycznych sieci lokalnych



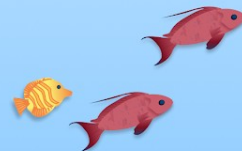
- W pierwszych tygodniach konfigurowaliśmy wirtualne sieci lokalne
 - W zwykłej sieci każdy komputer jest połączony poprzez jakieś fizyczne medium
 - miedź, światłowód, fale radiowe...
- Co jeżeli chcielibyśmy stworzyć sieć lokalną z komputerów odległych od siebie geograficznie?
- Co jeżeli chcielibyśmy dać pracownikom dostęp firmowej do sieci lokalnej z domu?



A jakby tak użyć IP jako „medium”?



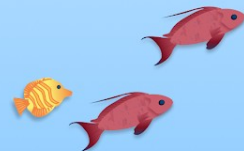
Czym jest VPN?



- **VPN – Virtual Private Network**
 - Virtual – bo nie ma fizycznych połączeń, zamiast tego używa istniejących sieci jako „warstwę fizyczną”
 - Private – powinna zapewniać uwierzytelnianie i szyfrowanie ruchu
 - W teorii są techniki takie jak IP in IP, albo IPSec bez szyfrowania, które tworzą wirtualne sieci bez zapewniania poufności – są to bardziej sieci tunelowane niż VPN



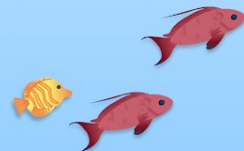
Czym jest VPN?



- Z perspektywy aplikacji – sieć jak każda inna
- W rzeczywistości pakiety skierowane do interfejsu VPN są szyfrowane i adresowane do odpowiedniego hosta w sieci fizycznej/internet, na odpowiedni port



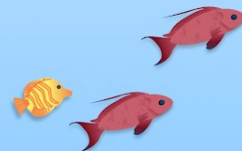
Protokoły VPN



- OpenVPN (UDP)
- WireGuard (UDP)
- IPSec (bezpośrednio na IP)
- Własne protokoły różnych firm
 - Cisco/Fortinet/GlobalProtect
- SSH w trybie tunelowania (TCP!)

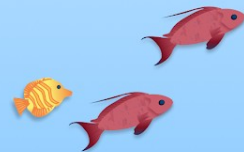


Problem z tunelowaniem przez TCP



- TCP zakłada, że sieć pod nim nie jest niezawodna i próbuje zapewnić niezawodność warstwom wyżej
 - ...co się stanie jeżeli na TCP damy kolejną warstwę TCP?
- TCP meltdown
 - Gdy fizyczna sieć zawiedzie, zarówno niższa jak i wyższa warstwa TCP zaczną retransmisję pakietów
 - ▶ Warstwa wyższa TCP nie wie, że warstwa niższa już próbuje zapewnić niezawodność
 - Warstwa wyższa TCP tworzy więcej pakietów dla warstwy niższej do przesłania
 - ▶ Doprowadzi to do zapchania warstwy niższej bezużytecznymi pakietami



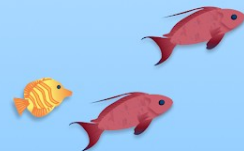


Rozdział 2

Jak działa Wireguard?



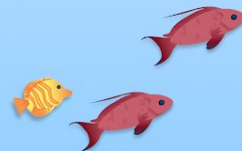
Dlaczego Wireguard?



- Mały i prosty
- Implementowany w kernelu = wydajniejszy niż implementacje userspaceowe
 - Od 2020 roku wspierany przez kernel Linux (wersja 5.6)
- Protokół P2P
 - Może tworzyć sieci mesh (ale też client-server)
 - Przy dobrej konfiguracji firewall go nie zatrzyma
- W miarę prosty w konfiguracji dla małych sieci
 - Niezbyt odpowiedni dla dużych sieci dostępowych



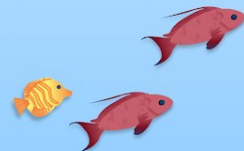
Jak działa Wireguard?



- Po utworzeniu interfejsu wireguard, konfigurujemy mu port UDP do nasłuchu
 - Jednocześnie jest to port źródłowy dla pakietów wychodzących
- Każdy uczestnik (interfejs) ma swój klucz prywatny, którym się uwierzytelnia
 - Każdy uczestnik ma klucze publiczne każdego skonfigurowanego uczestnika
 - Nie zawsze każdy ma każdego hosta skonfigurowanego jako uczestnik



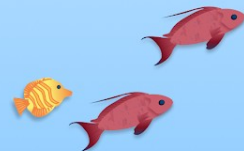
Jak działa Wireguard?



- Każde połączenie (para uczestników) może mieć skonfigurowany współdzielony klucz symetryczny
 - Zapobiega to potencjalnym przyszłym atakom z wykorzystaniem komputerów kwantowych
- Wireguard wykorzystuje klucze symetryczne do przesyłu faktycznych danych
 - Automatycznie ustalane w ramach „handshake”
 - Wymieniane co kilka minut



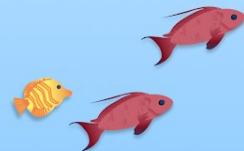
Jak działa Wireguard?



- Port UDP wysyła odpowiedzi tylko wtedy, gdy są podpisane przez skonfigurowanego uczestnika
 - = Wireguarda nie da się wykryć skanerami portów
 - (ale można innymi technikami, np. snifferami)
- Domyślnie Wireguard nie wysyła żadnych pakietów, jeżeli nie ma żadnych danych do przesłania
 - Można to zmienić, np. jeżeli uczestnicy są za NAT



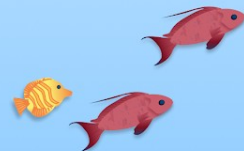
Jak działa Wireguard?



- Każdy uczestnik może (nie musi) mieć skonfigurowane adresy IP i porty innych uczestników
 - Co najmniej jeden uczestnik w każdej parze musi
 - To kto ma skonfigurowane adresy zależy od typu konfiguracji
 - Po otrzymaniu poprawnego pakietu od uczestnika, Wireguard automatycznie aktualizuje jego adres w konfiguracji interfejsu
 - Adresy internetowe użytkowników mogą się zmieniać bez potrzeby ponownego uruchomienia tunelu



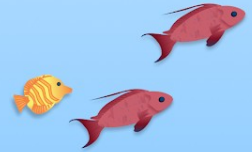
Jak działa Wireguard?



- Każdy uczestnik ma przypisane adresy IP, których może używać w VPN
 - Jeżeli Wireguard otrzyma pakiet od uczestnika, który nie jest uprawniony do korzystania z danego adresu IP, jest on odrzucany
 - Lista dozwolonych adresów IP jest wykorzystywana przy adresowaniu pakietów VPNowych do odpowiedniego hosta
 - Pakiet otrzymuje ten uczestnik, który jest uprawniony do adresu docelowego

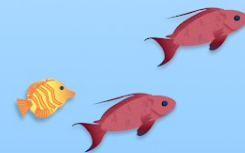


Jak działa Wireguard



- Na linuxie, Wireguard zawsze odbiera/wysyła pakiety z przestrzeni, w której został utworzony
 - Nawet gdy zostanie później przeniesiony do innej przestrzeni!
 - Można w ten sposób tworzyć przestrzenie, gdzie interfejs Wireguard jest jedynym interfejsem
 - ▶ = zmusić przestrzenie do przesyłania wszystkiego przez VPN



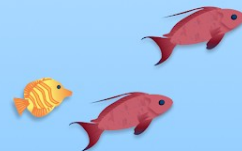


Rozdział 3

Konfigurowanie Wireguarda



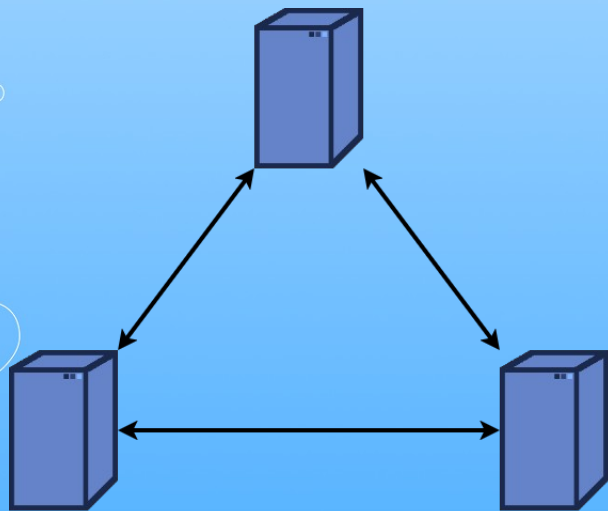
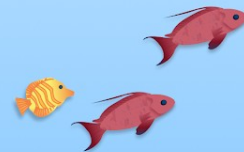
Zanim utworzysz VPN Wireguard...



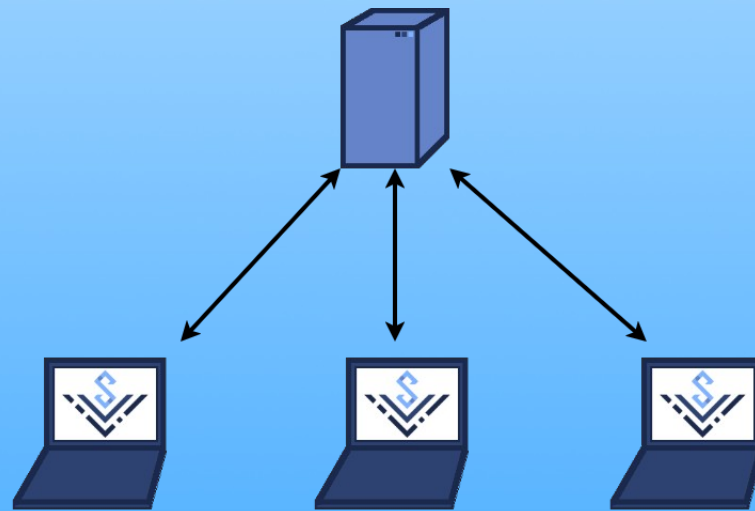
- **Jakiego typu sieć chcesz utworzyć?**
 - **Peer-to-peer/mesh?**
 - ▶ Każdy zna każdego, ma adres każdego
 - ▶ Dobrze gdy każdy uczestnik ma adres publiczny
 - Czyli najczęściej przy serwerach... albo IPv6 🙄
 - ▶ Najmniejsze opóźnienie, większa niezawodność – każdy pakiet jest adresowany bezpośrednio do celu
 - **Client-server?**
 - ▶ „Serwer”/gateway zna każdego, „klienci”/użytkownicy znają tylko serwer i autoryzują go do całego zakresu adresacji VPN
 - ▶ Dobrze gdy uczestników jest dużo i/lub się często zmieniają
 - ▶ Albo gdy użytkownicy są za (CG-)NATem
 - ▶ Duże opóźnienie, single point of failure – pakiet zawsze jest przesyłany przez serwer



Typy sieci



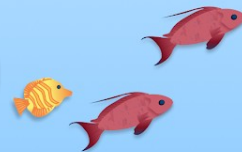
Mesh/Peer-to-peer



Client-server



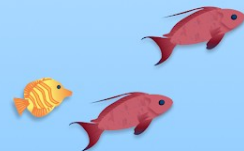
Sposoby na konfigurację Wireguard



- Wiele sposobów na konfigurację tego samego:
 - Ręczne przez komendy ip i wg
 - **Pliki konfiguracyjne + wg-quick**
 - systemd-networkd (.netdev + .network)
 - NetworkManager (graficznie)



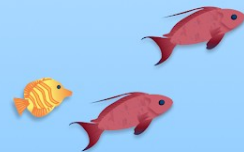
Przykład konfiguracji



```
1 [Interface]
2 # Lista adresów dla interfejsu lokalnego
3 Address = 10.0.0.1/32, 2001:41d0:602:1eea::1/32
4 # Port UDP do nasłuchu
5 ListenPort = 12345
6 # Nasz klucz prywatny
7 PrivateKey = [redacted]
8
9 # Nowy uczestnik
10 [Peer]
11 # Klucz publiczny
12 PublicKey = [redacted]
13 # Klucz symetryczny połączenia
14 PresharedKey = [redacted]
15 # Lista autoryzowanych adresów
16 AllowedIPs = 10.0.0.2/32, 2001:41d0:602:1eea::1:2/32
17 # Publiczny adres IP i port uczestnika
18 Endpoint = domena.tez.dziala.minibomba.pro:12345
19
20 # Uczestnik 2
21 [Peer]
22 # Klucz publiczny
23 PublicKey = [redacted]
24 # Bez klucza symetrycznego
25 # Lista autoryzowanych adresów
26 AllowedIPs = 10.0.0.3/32, 2001:41d0:602:1eea::1:3/32
27 # Nieznany adres - ustalony przy pierwszym połączeniu
28 # Zamiast tego ustawimy podtrzymanie połączenia
29 PersistentKeepalive = 30
```



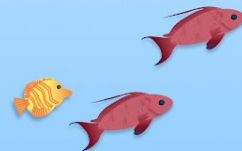
Jak generować klucze?



- wg genkey – generuje klucz prywatny
- wg pubkey – oblicza klucz publiczny
- wg genpsk – generuje klucz symetryczny



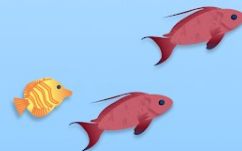
Uruchamianie konfiguracji



- Zapisać plik jako <nazwa interfejsu>.conf
 - Z reguły zapisywany w /etc/wireguard
- wg-quick up <plik konfiguracyjny>
- wg-quick down <plik konfiguracyjny> by usunąć



Konfiguracja P2P

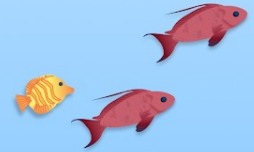


```
1 [Interface]
2 Address = 10.0.0.1/32
3 ListenPort = 12345
4 PrivateKey = [redacted]
5
6 [Peer]
7 PublicKey = [redacted]
8 # ten sam co po drugiej stronie
9 PresharedKey = [redacted]
10 AllowedIPs = 10.0.0.2/32
11 Endpoint = 2.2.2.2:12345
```

```
1 [Interface]
2 Address = 10.0.0.2/32
3 ListenPort = 12345
4 PrivateKey = [redacted]
5
6 [Peer]
7 PublicKey = [redacted]
8 # ten sam co po drugiej stronie
9 PresharedKey = [redacted]
10 AllowedIPs = 10.0.0.1/32
11 Endpoint = 1.1.1.1:12345
```

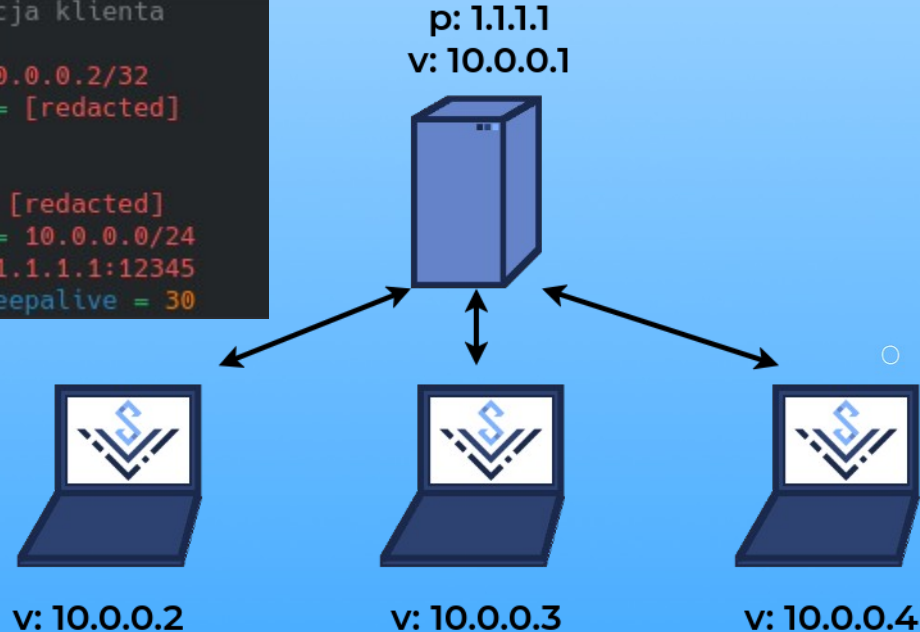


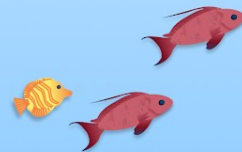
Konfiguracja C-S



```
1 # konfiguracja serwera
2 [Interface]
3 Address = 10.0.0.1/32
4 ListenPort = 12345
5 PrivateKey = [redacted]
6
7 [Peer]
8 PublicKey = [redacted]
9 AllowedIPs = 10.0.0.2/32
10
11 [Peer]
12 PublicKey = [redacted]
13 AllowedIPs = 10.0.0.3/32
14
15 [Peer]
16 PublicKey = [redacted]
17 AllowedIPs = 10.0.0.4/32
18
```

```
1 # konfiguracja klienta
2 [Interface]
3 Address = 10.0.0.2/32
4 PrivateKey = [redacted]
5
6 [Peer]
7 PublicKey = [redacted]
8 AllowedIPs = 10.0.0.0/24
9 Endpoint = 1.1.1.1:12345
10 PersistentKeepalive = 30
```



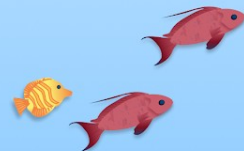


Rozdział 4

Wireguard vs stanowy firewall



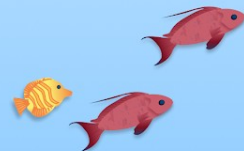
Stany firewall vs UDP



- Stany firewall zezwala na ruch, który jest częścią istniejącego połączenia
 - Bo często chcemy ograniczyć ruch przychodzący do wyłącznie zaufanych protokołów, a wychodzący zostawić bez ograniczeń
 - Firewall bezstanowy nie jest w stanie tego zrobić – może zablokować cały ruch przychodzący, wraz z odpowiedziami na ruch wychodzący
- ... ale UDP nie ma połączeń???
- Śledzenie połączeń w UDP jest... trochę „uproszczone”?
- UDP nie wysyła pakietów na początek i koniec połączenia
- Więc najczęściej zapamiętuje się porty na przesyłanych pakietach przez jakiś czas jako „aktywne połączenia UDP”



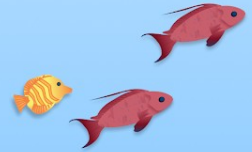
Stanowy firewall vs UDP



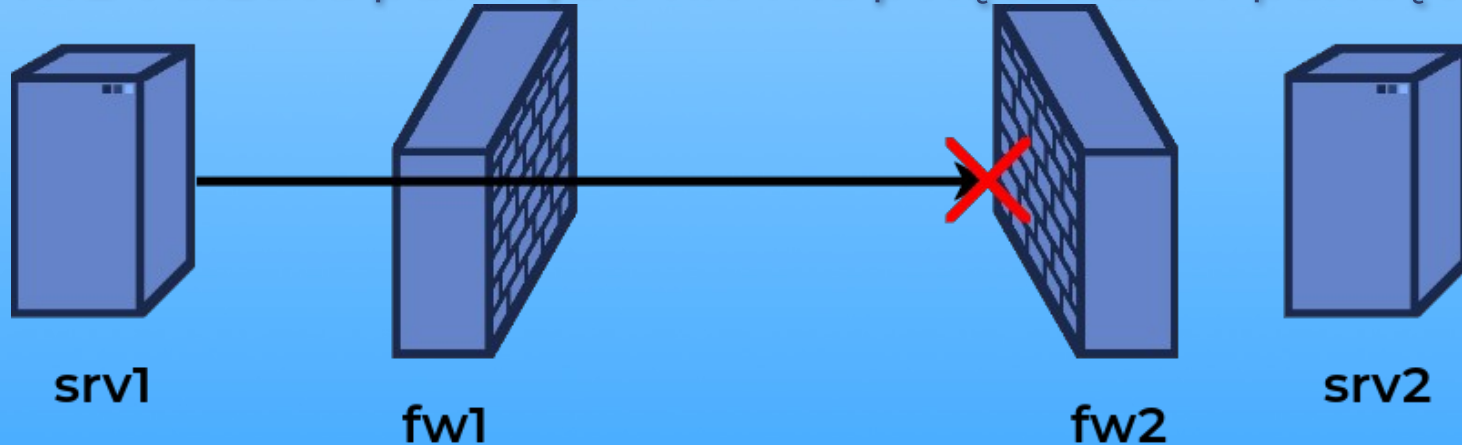
- Co jeżeli mamy protokół P2P, gdzie oba końce z jakiegoś źródła mają informacje o adresie IP i porcie drugiej strony?
 - np. ze statycznego pliku, lub od innego serwera
- Co jeżeli obaj uczestnicy będą próbować nawzajem nawiązać połączenie?
 - ...która strona będzie wg firewalla „połączeniem przychodzącym”?



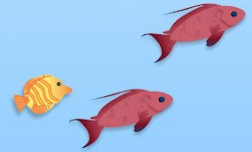
Stanowy firewall vs UDP



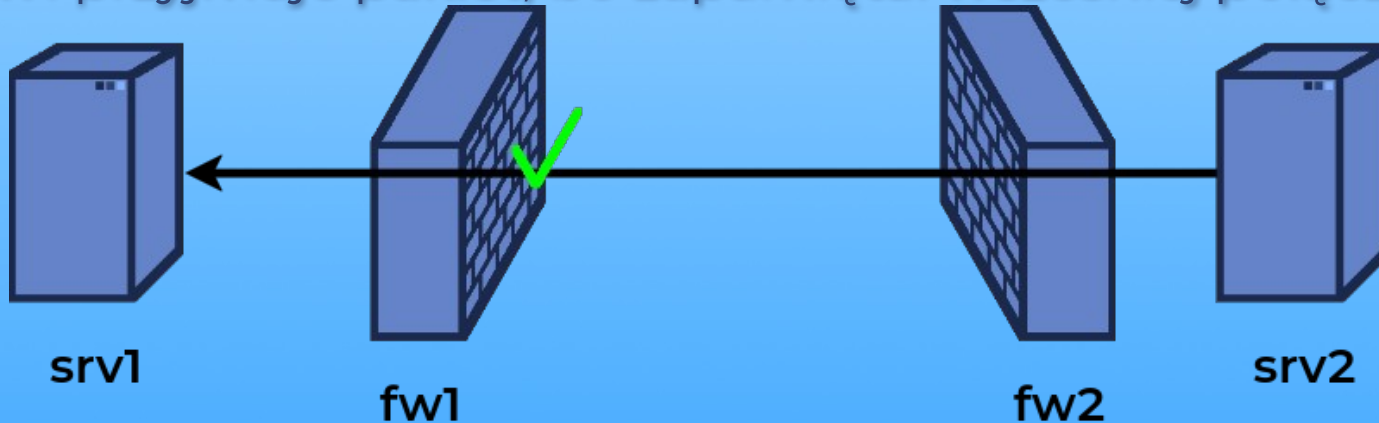
- Pierwszy pakiet (próba nawiązania z jednej strony)
 - fw1 zapamiętuje „połączenie”
 - fw2 odrzuca pakiet, bo nie ma połączenia w pamięci



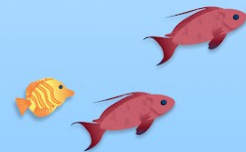
Stanowy firewall vs UDP



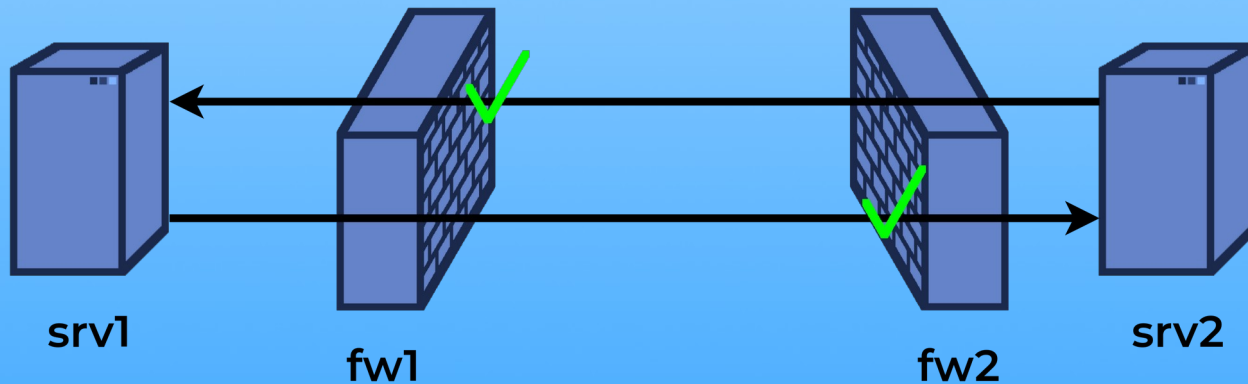
- Drugi pakiet (próba nawiązania z drugiej strony)
 - fw2 zapamiętuje „połączenie”
 - fw1 przyjmuje pakiet, bo zapamiętał wcześniej połączenie



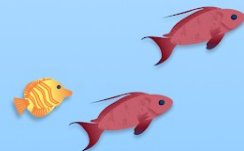
Stanowy firewall vs UDP



- Kolejne pakiety
 - Oba firewalle znają połączenie, oba przepuszczają pakiety



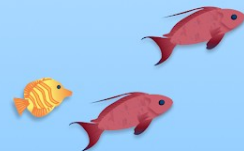
Stanowy firewall vs UDP



- Nie każdy ruch może w ten sposób omijać firewalla
 - Obaj uczestnicy muszą znać adres i port drugiej strony
 - ▶ Te informacje muszą być przewidywalne – CG-NAT może przeszkodzić
 - Obaj uczestnicy muszą próbować nawiązać połączenie
 - Obie strony muszą połączenie utrzymywać
 - ▶ Co jakiś czas wysyłać jakiś pakiet do siebie nawzajem
 - ▶ Inaczej firewalle mogą zapomnieć o połączeniu i zacząć je blokować



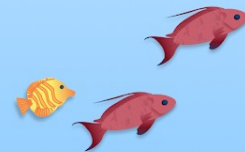
Stanowy firewall vs Wireguard

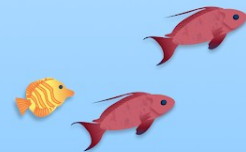


- By tak skonfigurować Wireguarda, po obu stronach:
 - Ustaw ListenPort= w sekcji [Interface]
 - Ustaw Endpoint= w sekcjach [Peer]
 - Ustaw PersistentKeepalive= w sekcjach [Peer]



Pytania?





**Dziękuję
za uwagę**

